

AnimeHub: La Next-Gen Community per Appassionati

Come l'ho implementato

AnimeHub non è un semplice database, ma un ecosistema digitale progettato per trasformare la visione passiva in un'esperienza sociale, analitica e gamificata.

Core Engine (Architettura Solida)

La base tecnologica di AnimeHub garantisce un accesso rapido e organizzato all'immenso catalogo mondiale:

- **Discovery Intelligente:** Home page con anime stagionali e titoli di tendenza.

Per l'integrazione dei dati relativi alle serie, la piattaforma si interfaccia con l'API GraphQL di AniList. Questo protocollo consente di ottimizzare il carico di rete effettuando query mirate, recuperando esclusivamente le informazioni necessarie (metadati, titoli, generi e copertine). Per disaccoppiare la logica di presentazione da quella di recupero dati, è stato implementato l'Angular Service `get-animes.ts`, che incapsula le chiamate API, gestisce le risposte del server e restituisce oggetti fortemente tipizzati pronti per l'uso nei componenti del frontend.

- **Ricerca Avanzata:** Sistema di filtri granulari per nome, genere, anno di produzione, formato (TV, Movie, OVA) e stato della serie per aggiungere titoli alle proprie liste.

La logica di filtraggio sfrutta il medesimo servizio `get-animes.ts`, al quale viene iniettato un oggetto di configurazione strutturato (`QueryOptions`) che incapsula tutti i criteri impostati dall'utente. Al fine di garantire la persistenza dello stato e la condivisibilità dei risultati, la pagina sincronizza i filtri attivi sia a livello di memoria di runtime, sia serializzandoli come Query Parameters all'interno dell'URL dell'applicazione; in questo modo, un eventuale refresh della pagina ripristina fedelmente i risultati filtrati.

Il sistema gestisce la paginazione lato server in modo disaccoppiato: le opzioni di ordinamento (sorting) e il contatore della pagina corrente sono stati separati dall'oggetto `QueryOptions` principale per consentire aggiornamenti mirati e reattivi dal sotto-componente `search-results`. L'architettura dell'interfaccia adotta un pattern Padre-Figlio: il componente `search-bar` (Figlio A) gestisce i controlli di input e propaga i filtri elaborati al componente Padre; quest'ultimo esegue la chiamata al servizio e distribuisce i dati al componente `search-results` (Figlio B), deputato alla renderizzazione delle

singole `AnimeCard` per l'accesso al dettaglio della serie, oltre che alla gestione dei controlli della paginazione.

- **Ecosistema Utente:** L'utente per sbloccare la **Vera** esperienza di AnimeHub, la creazione di **profili personalizzati** e **salvataggio dei progressi**, deve autenticarsi. È previsto pertanto un sistema di registrazione e di login, che permette una comunicazione sicura con il sistema mediante l'utilizzo di **JWT token**.

Il flusso di autenticazione adotta una strategia a doppio token per coniugare sicurezza ed esperienza utente: un **Refresh Token** a lungo termine viene memorizzato in modo persistente nel database sul server e associato all'account utente, oltre ad essere trasmesso al client tramite un cookie con flag `HttpOnly` (schermando così la sessione da vulnerabilità di tipo Cross-Site Scripting (XSS)). Parallelamente, un **Access Token** a breve termine viene trasmesso al client nel body della risposta per essere utilizzato in memoria dal frontend Angular per le normali chiamate API.

La gestione dello stato di autenticazione sul client è centralizzata nel servizio `auth-service`, il quale espone un Angular Signal reattivo denominato `user` che può assumere tre stati semantici ben definiti: `undefined` rappresenta la fase transitoria di bootstrap (caricamento della sessione in corso), `User` (oggetto tipizzato) certifica una sessione attiva e verificata, mentre `null` sancisce l'assenza di un utente loggato. All'avvio dell'applicazione, il metodo asincrono `initSession()` avvia il controllo preventivo dei token.

La persistenza e la sicurezza delle richieste HTTP verso il backend (porta `3001`) sono garantite da un `HttpInterceptorFn` che intercetta i flussi in uscita inserendo le credenziali necessarie. Lato server, un middleware dedicato valida i token: in caso di Access Token scaduto (risposta `401 Unauthorized`), l'interceptor del client congela temporaneamente la richiesta originale, effettua una chiamata trasparente all'endpoint di refresh per rinnovare la sessione e, in caso di esito positivo, riesegue la chiamata originale. Qualora anche il rinnovo fallisca, l'interceptor attiva automaticamente la procedura di logout forzato sul client.

L'utente autenticato potrà sfruttare le opzioni di tracking implementate attraverso la gestione di 3 liste private per organizzare degli anime (*Watching, Completed, Dropped*). La logica applicativa delle liste è orchestrata dal servizio specializzato `lists-service`, il quale espone i metodi per l'inserimento di una serie in uno dei tre stati previsti, la modifica dello stato stesso e l'avanzamento degli episodi. Il tracking delle puntate supporta sia l'incremento atomico (+1 click), sia la selezione di un episodio specifico che valida massivamente tutte le puntate antecedenti nel database. L'interfaccia utente centralizza queste interazioni all'interno della schermata dedicata `watchlist`.

- **Gestione Accessi (Angular Route Guards):** Implementazione di protezioni avanzate a livello di routing.

Le pagine sensibili (Profilo privato, Shared Lists) sono protette da AuthGuards, che

garantiscono che solo gli utenti autorizzati possano accedere ai propri dati, rendendo la piattaforma sicura e a prova di intrusioni. La sicurezza del routing è affidata a due Route Guards funzionali asincrone di Angular.

La guard `authGuard` protegge le aree strettamente riservate (area profilo, watchlist personali), monitorando lo stato del Signal `user`: la guard sospende la navigazione fintanto che lo stato è `undefined`, per poi concedere l'accesso o reindirizzare al `/login` non appena lo stato si stabilizza su `User` o `null`.

La seconda guard, `sharedListGuard`, estrae dinamicamente l'identificativo della lista dai parametri di rotta (`:listId`) ed effettua una chiamata preventiva al backend per verificare i diritti di membership dell'utente corrente; in caso di esito negativo o errore del server, l'accesso viene negato a monte e l'utente viene reindirizzato alla home.

Le Nuove Feature (Social & Dynamic)

Le ultime implementazioni elevano AnimeHub da semplice catalogo a **Social Network d'élite**.

1. Maratone Condivise & Feed Dinamico

La visione di gruppo diventa un'esperienza viva e pulsante attraverso le **Shared Lists**.

La shared list è una lista in cui più utenti possono entrare e segnare gli episodi visti negli anime condivisi.

- **Leaderboard:** Ranking basato su episodi visti e serie completate nel gruppo.
- **Social Interaction:** Titoli dinamici (es. *Sensei*, *Rivale*) per premiare i membri più attivi.
- **Roles:** Divisione in ruoli e permessi speciali per modificare la lista (es. Owner, Editor, Member).

La logica di autorizzazione all'interno delle liste condivise implementa un controllo degli accessi basato sui ruoli (RBAC). Il ruolo di **Owner** (creatore della lista) detiene il controllo amministrativo completo, potendo gestire l'anagrafica dei membri (inviti, rimozioni) e il catalogo della lista (aggiunta/rimozione serie).

Il ruolo di **Editor** possiede permessi limitati alla sola manipolazione del catalogo dei titoli, escludendo qualsiasi interazione con la gestione dei partecipanti.

Il membro con ruolo **Member** (utente standard) dispone esclusivamente dei permessi di lettura e di tracciamento dei propri episodi visti.

A livello di architettura dati, i progressi maturati all'interno di una lista condivisa non vanno a intaccare in modo distruttivo i dati della watchlist privata dell'utente. Al contrario, viene implementata una logica di allineamento unidirezionale: se l'utente incrementa il proprio

tracking all'interno della lista condivisa superando lo stato memorizzato nella propria area personale, il sistema aggiorna automaticamente anche il record privato, preservando la watchlist individuale come l'unica e assoluta "Source of Truth" dell'intero database dell'utente.

2. Dashboard Analitica

Ogni profilo diventa una "carta d'identità" estetica dell'appassionato, processando i dati in modo visivo:

- **Quick Resume:** Un carosello "Continua a guardare" per segnare i nuovi episodi visti con un solo click.

Per ottimizzare l'esperienza d'uso, questa funzionalità è stata strategicamente decentrata e sdoppiata in base al contesto: un'istanza è integrata nella pagina `watchlist-page` per gestire le serie personali in modalità *private*, mentre un'istanza parallela risiede nella pagina `shared-list-page` per tracciare le maratone di gruppo. Entrambi i componenti condividono la medesima logica reattiva: al clic sul controllo incrementale "+", il sistema aggiorna lo stato e, qualora l'episodio tracciato coincida con l'ultimo della serie, l'applicazione esegue in automatico la transizione dello stato dell'anime da *Watching* a *Completed*.

- **Data Visualization:** Analisi granulare dei generi preferiti e monitoraggio del *Watchtime* giornaliero tramite istogrammi precisi.

Il backend implementa dei trigger logici associati a ogni operazione di scrittura sul database SQLite3 (avanzamento episodi, completamento o eliminazione serie) per aggiornare in tempo reale le tabelle relazionali delle metriche utente. Al completamento di una serie, il sistema mappa e incrementa le occorrenze di tutti i generi associati a quel titolo all'interno del profilo statistico. Parallelamente, il tracciamento temporale computa la durata media dell'episodio per incrementare i contatori del tempo di visione giornaliero e totale, aggregando i dati sia dalle attività private che dalle liste condivise.

Nel caso in cui una serie venga rimossa dalla watchlist, il sistema esegue una compensazione sottrattiva sul tempo totale accumulato in base agli episodi precedentemente fruiti per quel titolo.

Questi dati vengono infine serviti a una tab analitica nel profilo utente, accoppiata a librerie di grafici per la visualizzazione di istogrammi e grafici a torta.

- **Time Tracking Assoluto:** Un contatore dettagliato del tempo totale speso nella visione (Giorni, Ore, Minuti), sincronizzato ad ogni episodio visto.

3. Social Architecture & Friendship System

A differenza dei tracker statici, AnimeHub implementa un'architettura di rete che permette la creazione di una community reale e interconnessa.

la creazione di una community reale e interconnessa:

- **Friendship Management:** Un sistema di gestione amicizie che permette di inviare inviti alle liste condivise.

La rete di relazioni è supportata dal servizio `friendship-service`, incaricato di gestire lo stato dei legami nel database (richieste inviate, accettate, lista amici). Questo livello è integrato direttamente con il servizio `shared-list-service`, il quale sfrutta la rete di amicizie dell'utente per abilitare le funzionalità di invito ai gruppi, espulsione (*kick*) dei membri e gestione dei livelli di privilegio (promozione/retrocessione dei ruoli amministrativi).

4. Gamification & Achievement System (Badge)

Un sistema di traguardi digitali:

Tier	Obiettivo	Esempio Badge
Bronzo	Inizio del percorso	<i>Newcomer</i>
Argento	Le prime conquiste	<i>Shonen Enthusiast</i>
Oro	Esploratore di generi	<i>Mecha Pilot</i>
Platino	500+ Serie completate	<i>Otaku Legend</i>

Badge Segreti: Traguardi "nascosti" per stimolare la curiosità e il passaparola nella community.

Il sistema di sblocco adotta un pattern ad eventi (*Event-Driven*): a seguito di ogni transazione sul database legata al volume di episodi visti o serie completate, il server interroga la tabella dei requisiti per verificare se l'utente soddisfa i criteri di un obiettivo.

Sul frontend, il client riceve la lista complessiva dei badge messi a disposizione dall'applicazione; un controllo condizionale applica una trasformazione CSS (filtro in scala di grigi) e un flag di opacità sui badge non ancora riscossi, fornendo un feedback visivo immediato sulla progressione dei propri traguardi rispetto a quelli collezionati.

Architettura dei Servizi (Frontend Service Registry)

Per garantire la massima modularità, la separazione delle responsabilità e la reattività dei dati tramite Angular Signals, la logica di business di AnimeHub è stata decentralizzata nei seguenti Angular Services:

1. AuthService (`auth-service.ts`)

È il pilastro della sicurezza dell'applicazione. Centralizza lo stato dell'utente e orchestra i

È il pilastro della sicurezza dell'applicazione. Centralizza lo stato dell'utente e orchestra i flussi di autenticazione asincroni.

- **user (Signal)**: Espone lo stato in tempo reale dell'utente connesso (`User` | `null` | `undefined`).
- **token (Signal)**: Memorizza l'Access Token a breve termine per le sessioni attive.
- **initSession()** : Esegue il bootstrap asincrono dell'applicazione al refresh della pagina, invocando il rinnovo dei token.
- **login(email, password) / register(...)** : Interfacciano il form di sottomissione credenziali con il backend per generare una nuova sessione.
- **refreshToken()** : Effettua la chiamata per rigenerare l'Access Token tramite il Refresh Token memorizzato nei cookie.
- **loadUser()** : Recupera i dati anagrafici del profilo dal server una volta convalidata la sessione.
- **logout()** : Invalida i token lato server e client, azzerando i Signals e reindirizzando l'utente alla rotta pubblica `/login` .
- **isAuthenticated()** : Metodo di utilità per verificare istantaneamente lo stato di validità della sessione.

2. **GetAnimesService (get-animes.ts)**

Gestisce l'astrazione dello strato dati e la comunicazione con il gateway GraphQL di AniList.

- **getSeasonalAnimes()** / **getTrendingAnimes()** : Interrogano il server per popolare i caroselli e le sezioni della Home Page.
- **searchAnimes(options: QueryOptions)** : Esegue query di ricerca avanzata e paginata, applicando i filtri granulari passati come argomento.
- **getAnimeDetails(id: number)** : Recupera la scheda tecnica dettagliata, la sinossi e i metadati di una specifica serie.

3. **AnimeDetails (anime-details.ts)**

Gestisce tutte le query verso AniList per ottenere i dettagli degli anime, tipo gli episodi, i personaggi e gli anime consigliati.

3. **ListsService (lists-service.ts)**

Orchestra le interazioni legate alla Watchlist privata dell'utente e memorizza la *Source of Truth* delle sue visioni individuali.

- **addAnime(animeStatus, animeDetails)** : Inserisce un nuovo titolo in una delle tre categorie d'ascolto (*Watching*, *Completed*, *Dropped*).
- **updateAnime(animeId, newStatus)** : Gestisce la transizione di stato di una serie (es. da *Watching* a *Completed*).

Watching a Completed).

- **addWatchedEpisode(animeId)** : Avanza atomicamente di un singolo episodio il tracciamento della serie (+1 click).
- **setEpisodesUpTo(animeId, episodeNumber)** : Convalida massivamente come "visti" tutti gli episodi antecedenti a quello selezionato.
- **removeAnime(animeId)** : Rimuove definitivamente una serie dalla Watchlist, invocando le logiche di compensazione statistica.

4. **SharedListService (shared-list-service.ts)**

Gestisce la logica collaborativa delle maratone di gruppo, il controllo degli accessi in base ai ruoli (RBAC) e la sincronizzazione unidirezionale con la lista privata.

- **createSharedList(name)** : Gestisce la creazione della lista condivisa.
- **addAnimeToSharedList(listId, animeDetails)** / **removeAnimeFromSharedList(listId, animeId)** : Funzionalità riservate a Owner ed Editor per manipolare il catalogo della stanza.
- **updateUserAnimeProgress(listId, animeId, episodeNumber)** : Registra il progresso del singolo utente all'interno del gruppo e, se superiore al progresso privato, innesca l'allineamento automatico.
- **inviteMember(listId, friendId)** / **removeMember(listId, userId)** : Funzionalità amministrative per la gestione dell'anagrafica dei partecipanti.
- **updateMemberRole(listId, userId, newRole)** : Permette all'Owner di promuovere o retrocedere un membro (es. da Viewer a Editor).

5. **FriendshipService (friendship-service.ts)**

Modella i nodi della rete sociale dell'applicazione, permettendo l'interconnessione tra i profili utente.

- **addFriend(friendId)** : Invia un invito di amicizia a un altro utente della community.
- **acceptFriend(senderUserId)** / **declineFriend(friendId)** : Gestisce le richieste in entrata modificando lo stato della relazione nel database.
- **getFriends()** : Recupera l'elenco tipizzato di tutti gli utenti con cui è attivo un legame di amicizia, sia confermato che in pending.
- **removeFriend(friendId)** : Interrompe il legame bilaterale tra due utenti.

6. **APIService (apiservice.ts)**

Rappresenta il client HTTP di basso livello, configurato per standardizzare le richieste verso il backend locale e astrarre i verbi HTTP principali.

- **get<T>(endpoint, options)** : Esegue richieste di lettura fortemente tipizzate.
- **post<T>(endpoint, body)** : Gestisce l'invio di payload JSON o di oggetti `FormData` (es. caricamento dell'avatar).

- `put<T>(endpoint, body)` / `delete<T>(endpoint)` : Standardizzano le operazioni di modifica e rimozione dei record sul server.

Si trovano anche le funzioni chiamate nella pagina dei dettagli degli anime per segnare in bulk le puntate viste.

Struttura del Server (Backend)

Il server è strutturato in tre parti principali (Rotte, Controller e Modelli) per dividere i compiti in modo ordinato:

- **Le Rotte (`Routes`)**: Sono le porte d'ingresso del server. Ricevono le richieste dal frontend Angular in base all'URL e, prima di far passare i dati, applicano i controlli di sicurezza (come la verifica dei token JWT).
- **I Controller (`Controllers`)**: Sono il motore logico del server e lavorano in modo speculare ai service del frontend. Gestiscono tutte le funzionalità dell'app prendendo i dati dalle rotte, chiedendo al database cosa fare e impacchettando la risposta finale da rispedire indietro.
- **I Modelli (`Models`)**: Si occupano esclusivamente del database (**SQLite3**). Parlano con i controller per eseguire concretamente le query (aggiungere un utente, aggiornare i minuti di visione, controllare le amicizie, ecc.).

Gestione dei File Statici

Tutte le immagini degli avatar caricati dagli utenti e le icone dei badge si trovano all'interno di una cartella sul server chiamata `/static`. Questa cartella è impostata come pubblica, così il frontend può accedere direttamente ai file e mostrare le immagini nell'app usando i normali URL.

Gestione dei dati

[Tabelle SQL](#)